TripGraph AI: An Agentic, Grounded System for Conversational Group Travel Planning

Ujjwal Rattan[a,*], Anantha Viswak A[a], Bharath Kannan M[a], Jeyachandran Alagesan[a], Suriya Prakash M[a], Siddaraju D. H.[a], Venturi Naveen[a], Abdul Hafiz[a] and Navneet Chandan[a]

[a]Indian Institute of Science, Bengaluru, India

Abstract. A single large language model can write a travel itinerary that reads well but is often wrong: it invents places, misjudges distances, breaks the budget, and ignores how tired a group will get. We argue that this task is better solved by an agentic architecture, and we build one. TripGraph AI turns a short group chat into a constraint aware, day by day itinerary drawn on a live map. Instead of one generative call, it runs a fixed graph of specialised agents in which a curated knowledge graph supplies real places and a set of live APIs ground every distance, price and forecast. A dedicated planning agent then selects, orders and paces the chosen activities over a real travel time matrix. We also introduce per role model routing, where heavy reasoning runs on a frontier model and lighter, high volume work runs on a free model, which removes about three quarters of the paid calls without hurting plan quality. We explain why the agentic decomposition is the right fit for this problem, how each part was built so that new data sources and capabilities can be added later, and we close the main text with the business outlook. The system is deployed with a public domain and HTTPS.

1 Introduction

Planning a group trip is a conversation that hides a hard constraint problem. A line such as "four days in Goa from Chandigarh, about INR 40k each, beaches and seafood, flying, group of four" fixes a budget, implies a transport mode, names a geography that has to be sequenced sensibly, and introduces a group whose energy has to be managed across the days. Handing the whole job to one language model is tempting, but it fails in four predictable ways. It invents attractions and hotels that do not exist. It guesses travel times and so packs days that cannot physically happen. It quietly loses the budget and returns plans that cost more than the cap. And it forgets the soft human factors, such as not stacking three long treks back to back. Every one of these is a point where a true fact or an actual calculation is needed, not more fluent prose.

This is why we treat the problem as an agentic one rather than a single prompt. The language model is kept for what it is genuinely good at, namely choosing among options, ordering them, and explaining the choices in natural language. Everything that has to be correct is delegated to components that can be checked: a curated knowledge graph for which places exist, and live services for distance, weather and price. The result is closer in spirit to retrieval augmented generation [2] and tool using agents [4, 3] than to free text generation, but it is deliberately organised as a fixed, inspectable pipeline rather than an open ended agent loop. We make four contributions. We present a deployed agent pipeline that fuses a knowledge graph with five live services (Section 2 and 3). We describe a planning agent that reasons over a real travel time matrix (Section 4). We give a configuration only mechanism for routing each agent to a different model (Section 5). And we set out the deployment and the business outlook (Sections 6 and 7). Supporting detail sits in the appendices and is referenced by number from here on.

* Corresponding Author. Email: ujjwalrattan@iisc.ac.in.

2 Why an agentic AI architecture

The case for an AI system here is straightforward. The input is open ended natural language, the output is a structured plan with reasoning attached, and the hard parts are selection and sequencing under soft preferences. Classical rule engines cannot read the chat, and a pure search planner cannot judge that a hill fort is worth an extra hour of driving. A language model supplies that judgement. The harder question is why the system should be *agentic* and not a single large prompt, and the answer shapes the whole design.

First, the task naturally decomposes into concerns that are easier to get right in isolation than together: understanding the request, finding candidates, grounding facts, building the schedule, pacing it, and checking it. Giving each concern its own agent means each can be prompted, tested and replaced on its own, and a weak step can be improved without disturbing the rest. Second, a decomposed pipeline can be grounded. Because retrieval and routing are separate agents, the planner is handed verified candidates and real travel times, so it can never place a city that does not exist or assume two stops are next to each other when they are an hour apart. Third, the pipeline degrades gracefully. If a live service is down, that single agent falls back to cached or seed data and the plan still completes, which a monolithic prompt cannot do. Fourth, an agentic graph is auditable: every field in the output can be traced to the node that produced it, which matters when a user asks why a stop was chosen or a budget was flagged.

It is worth being clear about what kind of agentic system this is, because the term covers two very different designs. One is a free running loop, in the style of ReAct [4], where a model decides at each step which tool to call next and stops when it judges itself done. That is powerful for open tasks, but for a product it is hard to predict: the number of tool calls, the cost and the latency all vary per request, and a wrong turn early on is hard to debug. We chose the other design, a fixed graph of agents with a known topology, because group trip planning has a stable shape. The stages are always the same, so we encode them once. This buys three things that matter for a deployed

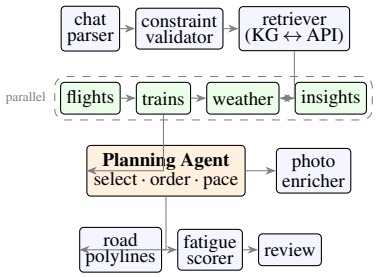

Figure 1. The agent pipeline. Blue nodes are structural, green nodes are concurrent grounding agents, and the orange node is the reasoning core. Extraction and validation precede a graph first retrieval; the planning agent works over grounded candidates; later agents enrich, pace and check the result.

service. Runs are reproducible, since the same input flows through the same nodes and yields the same plan. Cost is bounded, since the number of model calls is fixed rather than discovered at run time. And behaviour is debuggable, since a failure is localised to one node rather than buried in a chain of self directed decisions. We still get the benefits of LLM reasoning, but only at the nodes where reasoning is actually required.

The glue that holds the agents together is a single typed state object that every node reads from and writes a small patch back to. This shared contract is deliberate. It means an agent depends on the shape of the state, not on the internals of its neighbours, so agents can be developed and tested in isolation, reordered, or run in parallel when they do not depend on each other. It also makes the whole run observable: because each agent's contribution is an explicit patch, the pipeline can be traced end to end, which is how we attach run tracing and how a reviewer can see exactly which agent set a given field. Taken together, the typed state, the fixed topology and the per agent model choice are what let the project grow by addition rather than rewrite, which was a primary design goal from the start.

3 System architecture

TripGraph is a FastAPI backend that drives a LangGraph state graph [1], with a React and Vite map first frontend. Figure 1 shows the flow. The chat is first parsed into a typed constraints record. A validator then does light constraint reasoning: it spots conflicts, for example an expedition tier hotel against a low budget, and fills safe defaults for anything unstated while logging each assumption so the user can correct it before planning starts. A two pass retriever assembles candidate routes, hotels, activities and restaurants from the knowledge graph first, and only calls external services for whatever the graph cannot cover, caching every result to disk so repeat trips are near instant. Four grounding agents (flights, rail, weather and web insights) then run at the same time, on separate threads, so their network waits overlap. The planning agent consumes all of this, produces the day by day plan, and downstream agents enrich it with photos and real road geometry, score fatigue, and review it.

The knowledge graph is the part that keeps the system honest. It is a curated store of more than eighty Indian destinations, each with activities, hotels and transport options carrying coordinates, ratings, an indicative cost, a fatigue seed and a confidence score. The planner never invents a place; it selects from these verified candidates, which is what turns a believable plan into a plan whose every stop is real. The graph is read through a fallback layer that uses Neo4j when it is available and an equivalent JSON store otherwise, so the system has no hard database dependency and runs unchanged on a laptop or a small server.

Two design choices in the retrieval path are worth calling out, since both were made for the sake of later growth. The first is that retrieval is graph first and only reaches for a live service when the graph cannot answer, with every fetched record written back to a disk cache. This keeps the common path fast and free, and it means that adding a new live source later is a matter of filling one slot in the second pass rather than rewiring the planner. The second is that each candidate carries a confidence score, so the system can prefer verified, high confidence places now and, in future, blend in lower confidence discoveries from the web behind the same interface without the planner needing to know where a candidate came from. The grounding agents that run in parallel follow the same contract: flights and rail return structured advisories with indicative fares, weather returns a per day forecast that the planner turns into a gear note, and the insights agent returns short factual notes. None of them can block the run, because a failure in any one simply leaves its slot empty and the planner proceeds with what it has.

4 The planning agent

The planning agent is the reasoning core and the main place where real judgement happens. In one structured call it receives the constraints, the chosen transport mode and hotel, the candidate pool, a real driving time matrix $D \in \mathbb{R}^{n \times n}$ over the top candidates, the day by day weather, and short web notes. It returns a plan in which each day is a sequence of typed events (activity, meal, travel, rest, hotel), and each event carries a start time, a duration, a one line reason, fatigue and morale scores, a skippability rating and local tips.

Three rules keep the output usable rather than merely plausible. The agent inserts an explicit travel event between every pair of physical stops, with a mode and a traffic buffer sized from D, so the map shows the drive to the airport, the flight, and the drive to the hotel instead of a jump in space. Each day starts at the hotel and ends with a return leg, and arrival afternoons are filled with light activities rather than left blank. A hard cap of seven events per day, enforced in the prompt and by a post processor that trims only low priority stops, keeps days readable and bounds output length, which is the largest single driver of latency (Appendix C). The travel time matrix matters because estimating distance is exactly what a language model cannot do reliably on its own; handing it a cheap, real matrix removes the most common failure of LLM planners.

Pacing is handled by a fatigue agent that refines the seed scores for context. The adjusted fatigue of the kth event of a day is

$$\hat{f}_k \;=\; \mathrm{clip}_{[0,10]}\Big(f_k^{0} + \alpha\, c_k + \beta\, \mathbb{1}[\mathrm{rain}_k] + \gamma \sum_{j<k} \mathbb{1}[f_j^{0} \geq \tau] \Big), \quad (1)$$

where f_k^0 is the catalogue seed, c_k is the distance covered so far that day, the indicators add load for bad weather and for earlier intense stops, and $\alpha, \beta, \gamma, \tau$ are small fixed weights. The resulting scores drive the skippability rating the interface uses to mark which stops are essential and which can be dropped when a group is tired. Finally a review agent checks the plan for budget and constraint violations and either approves it or returns feedback for one bounded re plan, surfacing problems such as a budget overage to the user as an advisory rather than silently changing the plan.

5 Per role model routing

A single run issues many model calls of very different value. Parsing and validation are cheap and forgiving, while planning and review

need strong structured reasoning. We use this with per role routing: each agent asks a factory for its model, and the provider and model name for every role are read from configuration, so any agent can move between providers without touching code. Our deployed setup (Table 1) keeps the planner and review on a frontier model and sends the rest to a free, fast model. This dropped paid calls per plan from about eight to about two, roughly a seventy five percent cut, with no drop in plan quality, because the offloaded roles emit small, well specified JSON that the smaller model handles reliably. Independent roles also run on parallel threads. The mechanism is the same one that makes future integration cheap: a new model, including a local one for offline use, is a configuration change rather than a code change.

Table 1. Per role model routing in the deployed system.

Role(s)	Model class	Why
Planning agent	frontier	hardest reasoning
Review	frontier	quality gate
Parser, validator	free, fast	small forgiving JSON
Flights, trains, fatigue	free, fast	high volume
Explainer, enricher	free, fast	narrative and lookup

6 Map native interface and deployment

The plan is presented on the map, because a list of bullets hides the spatial reasoning that travellers care about. Each stop is a numbered pin in visiting order, joined by the real road polyline from the routing service, with inter city legs drawn as a dashed arc, so the route reads as it will actually be travelled. Selecting a pin opens a card pinned to that point on screen, with a live photo and rating, the planner's reason, opening hours, the fatigue gauges from Equation (1) and local tips. A day strip and a calendar timeline give the time view and are linked to the map both ways, and a cost ledger shows where the budget goes by category. A delay simulator lets the user inject a disruption and watch the day re plan with a plain language summary of what changed. The system is deployed behind a domain with HTTPS: an nginx proxy serves the built app and forwards the API to a backend run under systemd, provisioned by a single script. Every external call is wrapped so a failure downgrades instead of breaking the plan, chat input passes a prompt injection guardrail, and a maintenance flag can disable paid endpoints while keeping the site visible (Appendices E and F).

7 Future scope and business outlook

The same properties that make the architecture sound also make it a viable product. The clearest route to revenue is booking commission: the system already proposes specific hotels, flights and trains, so connecting the existing scaffolded deal and transport agents to live inventory turns every itinerary into a set of affiliate booking links at no extra modelling cost. A second route is business to business licensing. Travel agencies and online travel platforms can embed the planner as a white label service, where the per role routing keeps their running cost low because only two calls per plan touch a paid model. A third route is tiered access, offering free planning with a premium tier for larger groups, longer trips, live re planning during travel, and offline use through a local model. The opportunity is sized by a simple observation: group trips are common and high value, yet the planning step is still done by hand across chat threads, browser tabs and spreadsheets, and that friction is exactly what the system removes. A sensible go to market follows the architecture. The public demo validates the consumer experience and seeds the knowledge graph. From there the white label offering reaches travel agencies and regional operators who want a planner but cannot build one, and an API offering exposes the planning agent to other apps that already own a customer base. The unit economics are favourable because of the routing design: most of the work runs on a free tier, so the marginal model cost of an itinerary is small and predictable, which keeps the service viable even at consumer price points.

Defensibility comes from two places that are hard to copy quickly. The first is the curated knowledge graph, a compounding asset that improves selection quality as it grows with verified places and real bookings, and that a generic model cannot reproduce because it encodes choices and confidence, not just text. The second is the agentic structure itself, which lowers the cost of every future feature. Because each capability is an agent over a shared state, expansion to new regions, a corporate travel policy agent that enforces company rules, a group voting agent that turns shared decisions into a single plan, real time re planning during a trip, and a learned ranker for candidates can each be added as a node without reworking the pipeline. The product can grow feature by feature and market by market while the core stays stable, which is the commercial payoff of the engineering decisions described above. Extended details and a worked example follow in the appendices.

A Agent roster

Referenced from Section 3. The deployed pipeline comprises: chat parser (chat to constraints), constraint validator (conflicts and defaults), retriever (two pass graph and API with disk cache), mode planner (road, rail or flight by distance), planner orchestrator (route and hotel shell), terminal resolver (airports and stations with first and last mile times), flights and train agents (advisories with scraped fare ranges), weather agent (forecast to gear list), insights agent (web abstracts), the planning agent itself, photo enricher (place photos and ratings, parallel and capped), road polyline router (cached geometry), fatigue scorer and review agent. A refinement questioner powers an optional one round of counter questions, and a replanner agent powers the delay simulator. Deal and traffic agents are present as scaffolds for future live integrations.

B External data sources

Referenced from Section 3. Table 2 lists the live services. Graph first retrieval means most runs touch only a few of them. The routing service has the tightest free ceiling at two thousand requests per day, so per segment road geometry is cached aggressively.

Table 2. Live services and their role.

Service	Role
OpenRouteService	geocoding, routes, road polylines
Google Maps Platform	places (photos, ratings), distance matrix, tiles
OpenWeatherMap	day by day forecast
RailRadar	alternative rail routes
DuckDuckGo	destination notes, fare snippets

C Cost and latency

Referenced from Sections 4 and 5. The workload is bound by input and output, not computation: on a one vCPU cloud instance the server CPU stays below ten percent during planning, so the time is spent waiting on model tokens and API round trips, and vertical scaling does not help. Planner output length is the largest single term, which is why the seven events per day cap matters. Table 3 gives observed end to end latency. Per role routing cut frontier calls per plan from about eight to about two; the offloaded model sits on a free tier, so the marginal model cost of a typical plan is close to those two retained calls.

Table 3. Indicative end to end latency on a one vCPU instance.

Trip	Events	Wall clock
2 day weekend (warm cache)	about 12	about 30 s
4 day trip (cold cache)	about 30	about 140 s

D Worked example

Referenced from Section 4. For “Chandigarh to Goa, four days, INR 40k per person, flying, group of four” the planner picks a flight mode, opens day one with the cab to the airport, the flight, and the cab to the hotel, then a light arrival afternoon. Days two and three cluster the North Goa beaches and Old Goa heritage with explicit cab legs between stops and a sunset and seafood evening, and day four returns home. Each stop carries a reason, a fatigue and morale pair, and tips, and the review agent flags any budget overage as an advisory rather than exceeding it silently.

E Guardrails and parsing robustness

Referenced from Sections 4 and 6. Chat is treated as untrusted input. A prompt level instruction and a server side sanitiser reject injection patterns such as “ignore previous”, role markers and script or HTML, clamp numeric ranges, and flag off topic input. On parsing, a tolerant decoder strips code fences and reads the first valid JSON object even when the model adds trailing prose, which removed a class of silent fall backs to a template that appeared when we switched model families.

F Deployment topology

Referenced from Section 6. One script installs dependencies, builds the frontend, runs the backend under systemd, and configures nginx to serve the static assets and proxy the API, so the browser sees a single origin and CORS is not an issue. A second script provisions a Let’s Encrypt certificate and rebuilds the frontend against the secured origin. Secrets live only in an untracked environment file, and the maintenance flag returns an HTTP 503 from paid endpoints while leaving the landing experience in place.

References

[1] LangChain. LangGraph: Building stateful, multi-actor applications with LLMs. https://github.com/langchain-ai/langgraph, 2024.

[2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[3] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[4] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations (ICLR)*, 2023.